

# 实验三：垂直领域模型微调实践

## 1. 实验目标

在大模型实践中，全参数微调对算力要求极高。本实验旨在通过实践，让学生掌握参数高效微调（PEFT）的核心技术，特别是 LoRA（Low-Rank Adaptation）算法。学生需基于 Hugging Face 生态系统，完成一个特定垂直领域模型的指令对齐与微调部署，理解低秩矩阵对模型参数更新的模拟过程。

## 2. 实验核心任务

- **领域数据集构建**：学习将非结构化数据转化为“指令-输入-输出”标准格式，并应用正确的 Prompt Template。
- **LoRA 适配器配置**：调用 peft 库，配置 Rank、Alpha 及 Target Modules，理解其压缩参数的原理。
- **量化微调实践 (QLoRA)**：启用 4-bit 量化加载技术，在有限显存 (<16GB) 下实现 7B 级以上模型微调。
- **模型合并与部署**：掌握 Adapter 权重的保存、合并及基于推理框架的简易 Web 交互展示。

## 3. 实验指南

### 第一步：数据工程与 Tokenization

- **任务**：准备并加载垂直领域数据集。
- **关键点**：使用 datasets 加载 Json 格式数据；应用模型特定的模板（如 Llama-3/Qwen 模板）；处理 labels 掩码（Mask），确保 Loss 计算仅针对回答部分。

### 第二步：模型量化与 PEFT 注入

- **任务**：加载基座模型并配置 LoRA 适配器。
- **关键点**：使用 BitsAndBytesConfig 进行 4-bit 量化；定义 LoraConfig 并通过 get\_peft\_model 包装。

### 第三步：显存与微调效率观测（实验重点）

要求对比记录以下指标：

1. **参数规模**：记录可训练参数与原始总参数的占比。
2. **显存峰值**：对比开启量化与不开启量化时的显存开销。
3. **收敛特征**：记录 Loss 随训练步数的变化，观察垂直领域知识注入的稳定性。

#### 第四步：模型验证与交互展示

- **逻辑一致性**：验证微调后模型在垂直领域问题上的回答准确性及格式。
- **Web 展示**：编写 gradio 脚本，实现交互式对话。

## 4. 实验提交要求

学生需提交实验报告 PDF 及核心代码压缩包：

- **实验配置**：明确基座模型型号、LoRA 超参数 (  $r$ ,  $\alpha$  ) 及学习率。
- **结果展示**：包含训练 Loss 曲线图及 **Case Study 对比表** ( Base vs. LoRA-Tuned )。
- **分析思考**：讨论 LoRA 秩 ( Rank ) 的大小对模型拟合能力与训练显存的影响。

## 5. 关键代码示例

### (1) LoRA 配置与模型包装

Python

```
1  from peft import LoraConfig, get_peft_model, TaskType
2  from transformers import AutoModelForCausalLM, BitsAndBytesConfig
3
4  # 1. 配置 4-bit 量化
5  bnb_config = BitsAndBytesConfig(
6      load_in_4bit=True,
7      bnb_4bit_compute_dtype="float16",
8      bnb_4bit_quant_type="nf4"
9  )
10
11 # 2. 加载基座模型
12 model = AutoModelForCausalLM.from_pretrained(
13     "model_path_or_id",
14     quantization_config=bnb_config,
15     device_map="auto"
16 )
17
18 # 3. 配置 LoRA 适配器
19 lora_config = LoraConfig(
20     r=16,
21     lora_alpha=32,
22     target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
23     lora_dropout=0.05,
24     bias="none",
25     task_type=TaskType.CAUSAL_LM
26 )
27
28 # 4. 注入 LoRA 插件
29 model = get_peft_model(model, lora_config)
30 model.print_trainable_parameters()
```

## (2) 训练器启动 (Trainer)

Python

```
1  from trl import SFTTrainer
2  from transformers import TrainingArguments
3
4  training_args = TrainingArguments(
5      output_dir="./output",
6      per_device_train_batch_size=4,
7      gradient_accumulation_steps=4,
8      learning_rate=2e-4,
9      logging_steps=10,
10     max_steps=200, # 实验演示通常设置较短步数
11     save_strategy="steps",
12     save_steps=100
13 )
14
15 trainer = SFTTrainer(
16     model=model,
17     train_dataset=dataset,
18     args=training_args,
19     dataset_text_field="text", # 根据数据集字段调整
20     max_seq_length=512
21 )
22
23 trainer.train()
```

## (3) 模型推理与界面

Python

```
1  import gradio as gr
2
3  def predict(message, history):
4      # 此处省略 Tokenizer 处理流程
5      response = model.generate(**inputs)
6      return response
7
8  gr.ChatInterface(predict).launch()
```